

GEOG4/6300

Working with Raster and Vector Data:
Visualizing and zonal statistics

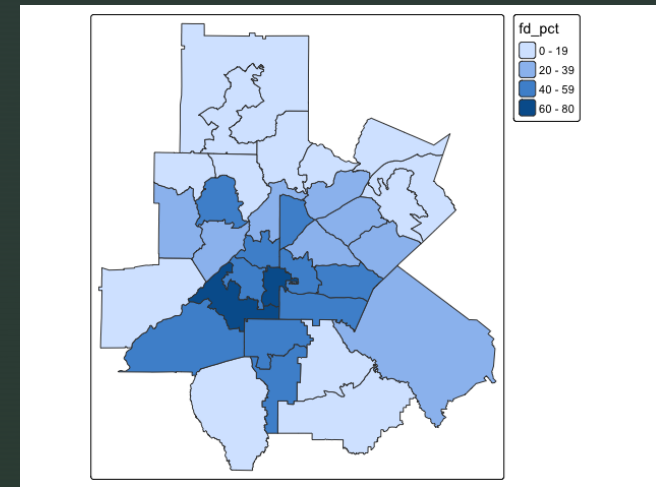
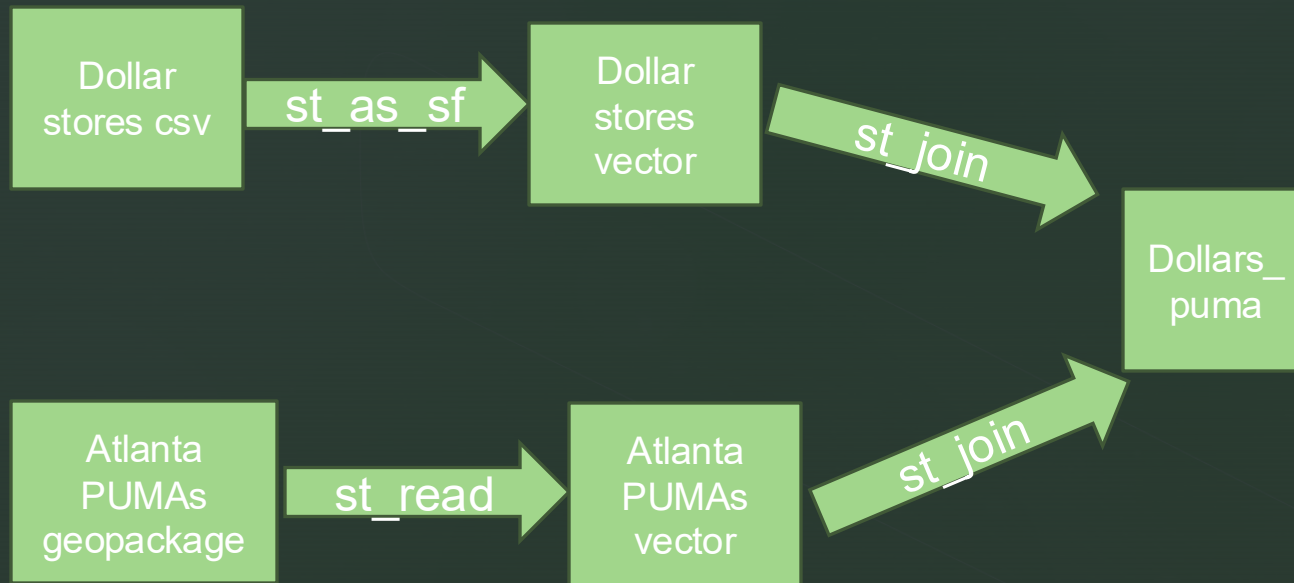
Tuesday September 30, 2025

Dr. Anna Harper

Overview

- Where are we in the class ...
- Review of calculating zonal statistics with vector data
- Calculating zonal statistics from raster data

How did we calculate zonal statistics with vector data (on Tuesday)?

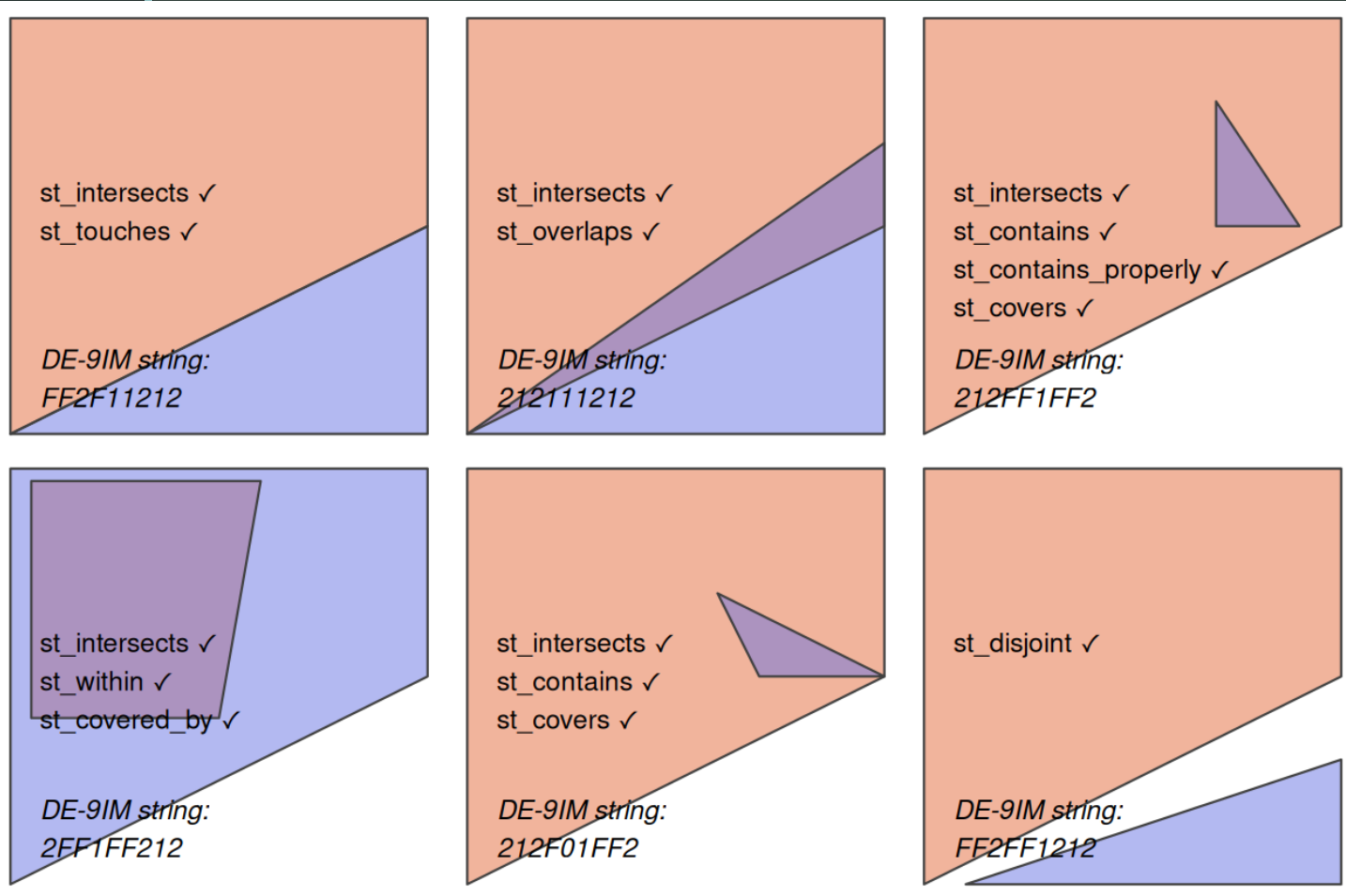


Joins

```
dollars_puma <- dollars %>%  
  st_join(atl_pumas %>% st_transform(4326), join=st_within) %>%  
  filter(is.na(GEOID)==FALSE)
```

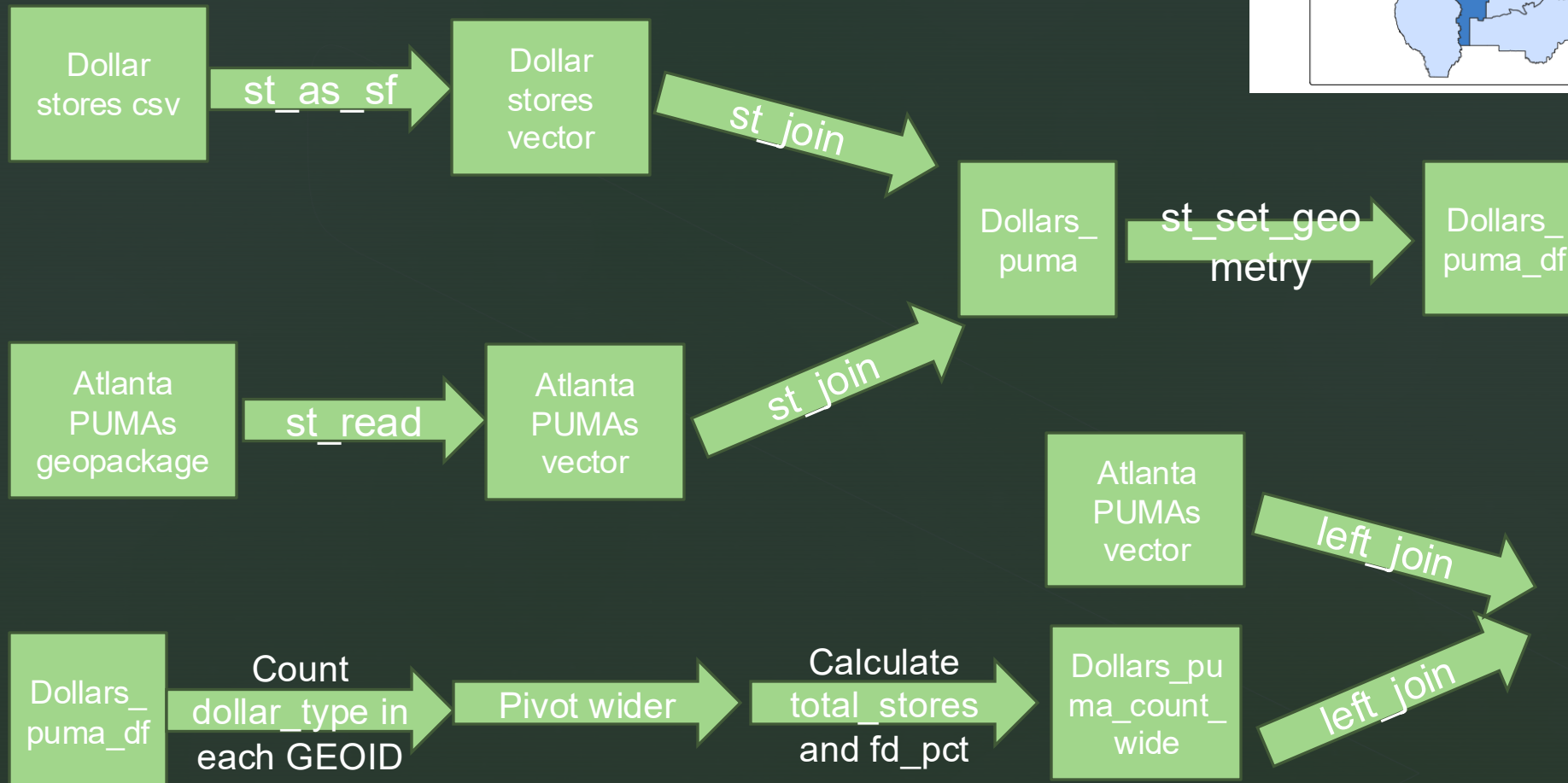
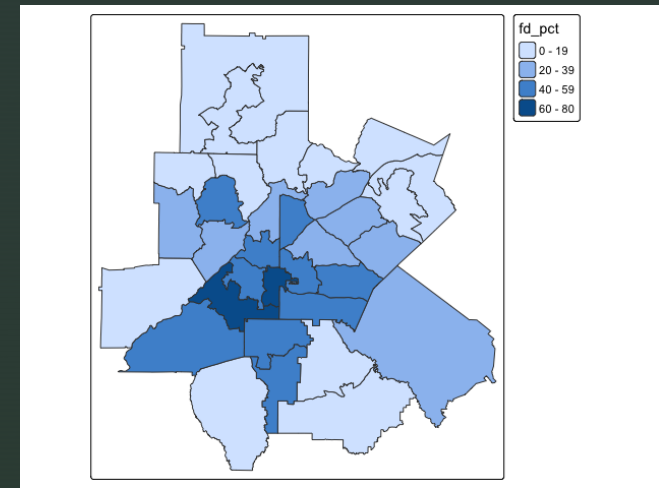
- *st_join* is a spatial join.
- The %>% after *dollars* means to use *dollars* as the first argument for *st_join*.
- ?st_join
- *st_within*: matches any feature in the first object (*dollars*) that falls completely inside a feature of the second object (*atl_pumas*).
- Don't return any stores that don't fall within one of the GEOIDs

Topological relations



- Describes the spatial relationships between objects
- `join(x,y)`: Order matters for within and contains
- Here: x is pink and y is blue

How did we calculate zonal statistics with vector data (on Tuesday)?

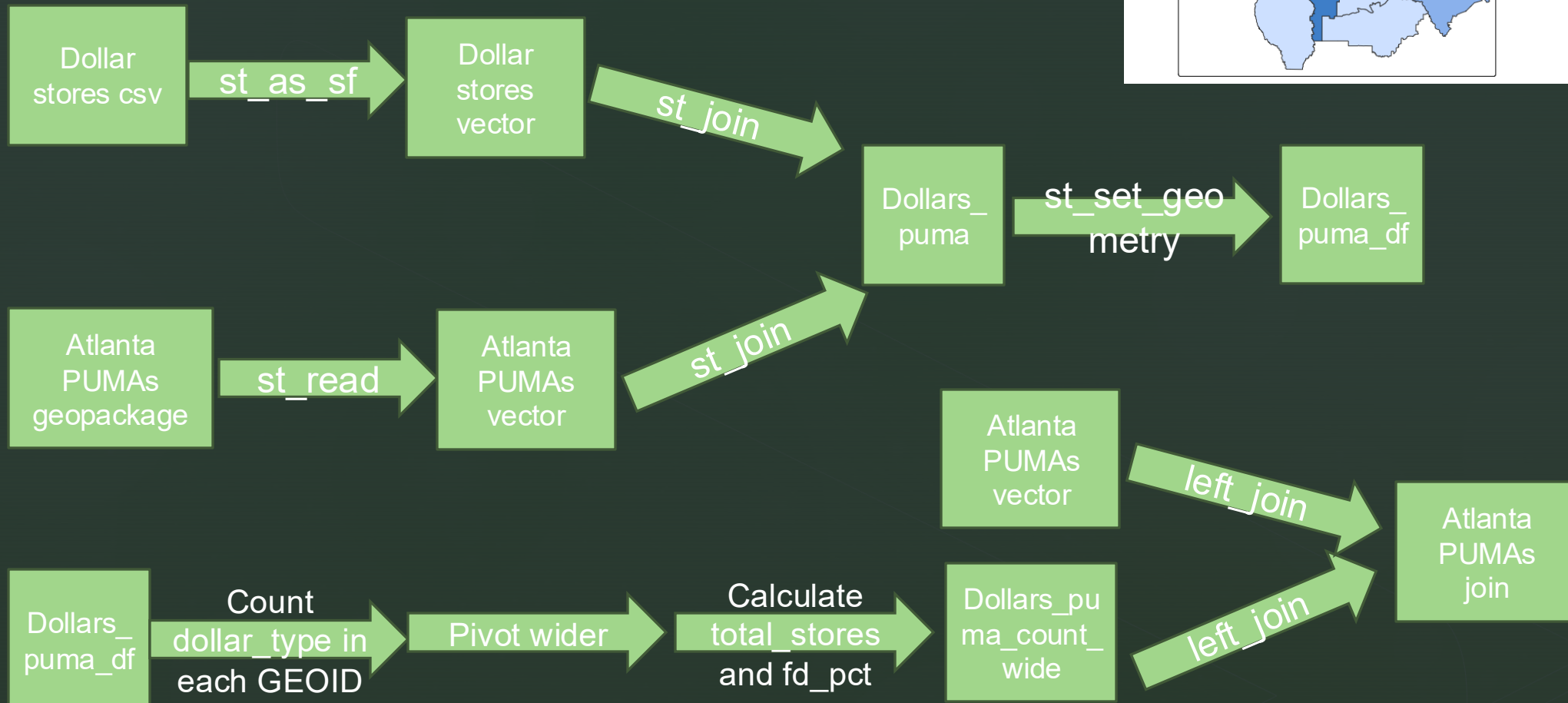
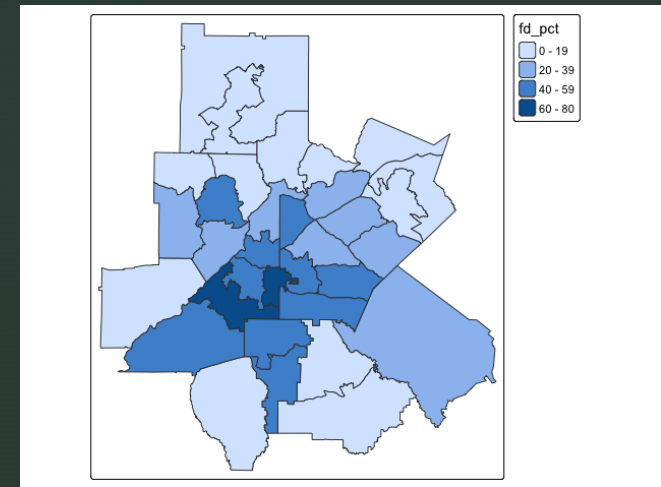


Joins

```
atl_pumas_join<-atl_pumas %>%  
  left_join(dollars_puma_count_wide)
```

- Review S02-2 if you're confused about joins, or week 2 content
- https://mikoontz.github.io/data-carpentry-week/lesson_joins.html
- Mutating joins add columns from y to x, matching observations based on the keys. There are four mutating joins: the inner join, and the three outer joins.
- A left_join() keeps all observations in x.

How did we calculate zonal statistics with vector data (on Tuesday)?



Further notes on vector manipulation

- *Filter* to subset rows (e.g. only choose Georgia in census data)
- *Select* to subset columns (e.g. only choose the columns with data you need)
- These both preserve the spatial component in sf objects.
- Using \$ removes the geometry data
- data[i, j]: i rows, j columns
- data[1:3, :]
- data[5, 1:5]

The standard set of comparison operators can be used in the `filter()` function, as illustrated in Table 3.1.

Symbol	Name
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code>></code> , <code><</code>	Greater/Less than
<code>>=</code> , <code><=</code>	Greater/Less than or equal
<code>&</code> , <code> </code> , <code>!</code>	Logical operators: And, Or, Not

TABLE 3.1: Comparison operators that return Boolean (true/false) values.

Chaining commands with pipes

- `%>%`
- As of R 4.1.0, you can also use `|>`
- The output of a previous function becomes the first argument of the next function, enabling *chaining*.
- The next two are equivalent:
 - `georgia_census <- filter(census_data, st_name == 'Georgia')`
 - `georgia_census <- census_data |>`
`filter(st_name == 'Georgia')`

See example in Script 07-1



Spatial operations on vector data

- It is important to note that spatial operations that use two spatial objects rely on both objects having the same coordinate reference system.

For more info, see
Lovelace Ch 4!

Stats with raster data

- *Local* or per-cell operations: Step 1
- *Focal* or neighborhood operations. Most often the output cell value is the result of a 3 x 3 input cell block
- *Zonal* operations are similar to focal operations, but the surrounding pixel grid on which new values are computed can have irregular sizes and shapes
- *Global* or per-raster operations. That means the output cell derives its value potentially from one or several entire rasters

Terra

- Global: "max", "min", "mean", "sum", "range", "rms" (root mean square), "sd", "std" (population sd, using n rather than n-1), "isNA", "notNA", "anyNA", "anynotNA"; or a proper R function (but these may fail for very large SpatRasters unless you specify maxcell)